

Time Complexity Analysis of the Euclidean Algorithm

Ashutosh Dash

July 30, 2026

1. C Source Code

The following is the exact implementation of the Euclidean algorithm used to calculate the Greatest Common Divisor (GCD), alongside the performance benchmarking routine:

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <time.h>
4
5 int Find_GCD(int a,int b)
6 {
7     if(a<b)
8     {
9         int c=a;
10        a=b;
11        b=c;
12    }
13    int c=1;
14    while(c>0)
15    {
16        c=a%b;
17        if(c==0)
18        {
19            return b;
20        }
21        a=b;
22        b=c;
23    }
24    return b;
25 }
26
27 int main()
28 {
29     srand(time(NULL));
30
31     int itr=1000000;
32     int *A = malloc(itr * sizeof(int));
33     int *B = malloc(itr * sizeof(int));
```

```

34     for(int i=0;i<itr;i++)
35     {
36         A[i]=rand();
37         B[i]=rand();
38     }
39
40     clock_t start = clock();
41     for(int i=0;i<itr;i++)
42     {
43         Find_GCD(A[i],B[i]);
44     }
45     clock_t end = clock();
46
47     double total_time = (double)(end - start) / CLOCKS_PER_SEC;
48     double average_time = total_time / itr;
49     printf("Number of iterations : %d\n", itr);
50     printf("Total execution time : %f seconds\n", total_time);
51     printf("Average time/GCD      : %f seconds\n", average_time);
52
53     return 0;
54 }

```

2. Time Complexity Proof

The time complexity of the `Find_GCD(a, b)` function is mathematically bounded by $\mathcal{O}(\log(\min(a, b)))$. The overall time complexity of the entire program (including the `main` loop) is $\mathcal{O}(N \log M)$, where N is the number of iterations and M is the maximum possible value generated by `rand()`.

Below is the step-by-step mathematical breakdown proving this boundary.

2.1 The Mechanics of the Loop

In each iteration of the `Find_GCD` function, the following assignments occur:

1. $c = a \pmod{b}$
2. $a = b$
3. $b = c$

This indicates that in the subsequent iteration, the new value of a takes the old value of b , and the new value of b takes the old value of $a \pmod{b}$.

2.2 Bounding the Remainder

To establish logarithmic time complexity, we must prove that the remainder $a \pmod{b}$ is always strictly less than $\frac{a}{2}$. This can be proven by evaluating two distinct cases:

- **Case 1:** $b \leq \frac{a}{2}$
By definition, the remainder of any division is always strictly less than the divisor. Therefore, $a \pmod{b} < b$. Since we assumed $b \leq \frac{a}{2}$, it mathematically guarantees that $a \pmod{b} < \frac{a}{2}$.
- **Case 2:** $b > \frac{a}{2}$
If b is strictly greater than half of a , then b can only fit into a exactly **1** time. The remainder is simply the difference: $a - b$. Because $b > \frac{a}{2}$, subtracting it from a leaves a result strictly less than $\frac{a}{2}$. Therefore, $a \pmod{b} < \frac{a}{2}$.

2.3 Complexity of the GCD Function

A classical result due to Lamé states that the Euclidean algorithm attains its worst-case running time when applied to consecutive Fibonacci numbers. However, we can also prove the logarithmic bound intuitively by tracking the halving behavior over two steps.

Suppose the loop begins with the values (a, b) . After one iteration, the new values are updated via the modulo operation:

$$(a, b) \rightarrow (b, a \pmod{b})$$

From our previous bounding observation, we know that $a \pmod b < \frac{a}{2}$.

During the second iteration, the variables are updated again:

$$(b, a \pmod b) \rightarrow (a \pmod b, b \pmod{a \pmod b})$$

At this point, the first variable has become $a \pmod b$. Since $a \pmod b < \frac{a}{2}$, the larger argument has been mathematically reduced by at least half within two iterations.

Since the larger number is reduced by at least a factor of two every two iterations, the total number of iterations is proportional to $\log_2(\min(a, b))$. In Big- $\mathcal{O}$ notation, we drop the constants, yielding a worst-case time complexity of:

$$\mathcal{O}(\log(\min(a, b)))$$

2.4 Total Program Complexity

- The `main` function loops `itr` times. Let $N = 1,000,000$.
- Inside this loop, `Find_GCD(A[i], B[i])` is executed.
- The values of `A[i]` and `B[i]` are constrained by the maximum limit of `rand()` (typically `RAND_MAX`). Let this maximum upper bound be M .
- Consequently, the worst-case execution time for a single GCD call is $\mathcal{O}(\log M)$.

Multiplying the cost of a single operation by the N loop iterations results in a total program mathematical time complexity of:

$$\mathcal{O}(N \log M)$$

References

- [1] *Data Structures and Algorithms (DSA) Class Notes*.
- [2] L^AT_EX code generated through Gemini AI.